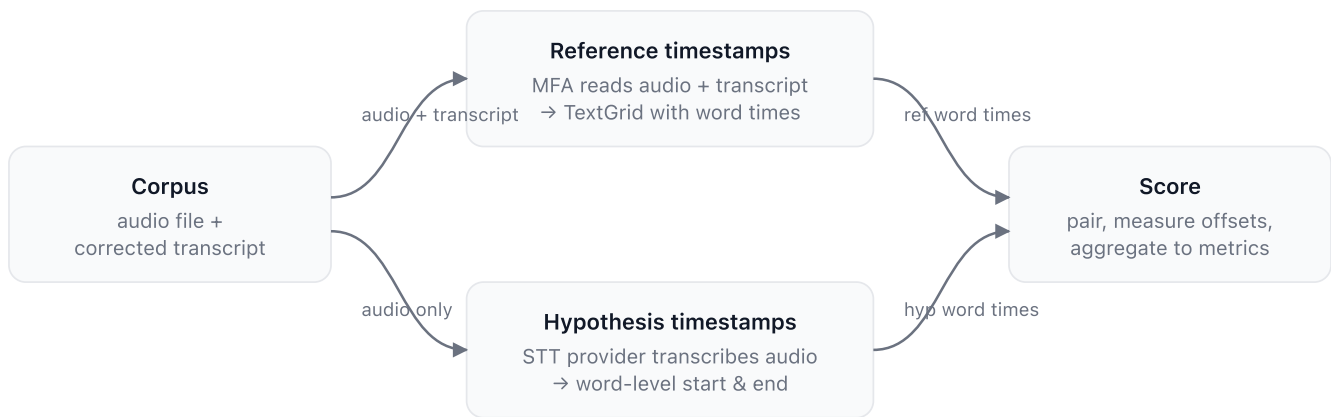


How this eval works

This page explains the eval methodology in plain language with concrete examples — what the harness does, what each metric means, what the results can and can't tell you, and where the inevitable noise comes from.

The pipeline at a glance



Two streams of word-level timestamps for the same audio — one from a forced aligner that "cheats" by being given the correct transcript, and one from the STT provider doing recognition from scratch. We line up the words and measure how far apart the times are.

A worked example, end to end

Let's walk through one real LibriSpeech utterance through every stage. The audio is 1.2 seconds long; the spoken transcript is:

"he doesn't work at all"

1. Reference from MFA

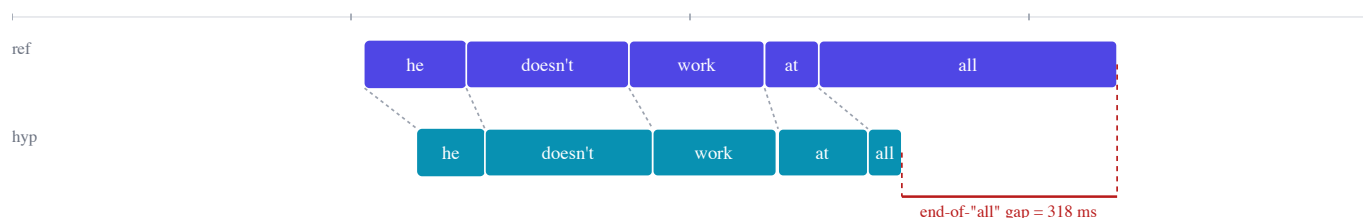
MFA takes the audio plus the correct transcript and aligns the two: it figures out exactly when each word's phonemes occur in the audio. The output is a TextGrid with one timestamped box per word. Drawn on a time axis:



■ MFA reference (this is the "ground truth")

2. Hypothesis from the STT provider

The same audio goes to the STT provider (here, AssemblyAI universal-3-pro). The provider produces its own word-level timestamps without seeing the ground-truth transcript. Drawn on the same time axis:



■ MFA reference ■ AAI hypothesis ■ notable boundary gap

Most word boundaries line up tightly. The exception is the end of "all" — MFA includes the trailing breath/silence in the word; the provider cut the word off earlier. **This isn't necessarily a model error** — it's a real disagreement about where the word ends. That ambiguity is intrinsic to word boundaries in continuous speech.

3. Aligning the two word sequences

Both sides have 5 words and they're in the same order, so pairing is straightforward: he ↔ he , doesn't ↔ doesn't , and so on. When the model mis-recognises words or the word counts differ, we use **minimum edit distance** alignment (the same algorithm WER uses) to find the best pairing. Substitutions still produce paired words — we score the timestamps regardless of whether the recognised text was right.

4. Measuring the offsets

Word	Ref start	Hyp start	Δ start	Ref end	Hyp end	Δ end
he	520	597	+77 ms	670	697	+27 ms
doesn't	670	697	+27 ms	910	946	+36 ms
work	910	946	+36 ms	1110	1129	+19 ms
at	1110	1129	+19 ms	1190	1262	+72 ms
all	1190	1262	+72 ms	1630	1312	-318 ms

Δ is the absolute offset: how far off the provider's word boundary is from MFA's. Sign just tells us "early vs. late." For metrics we take the absolute value.

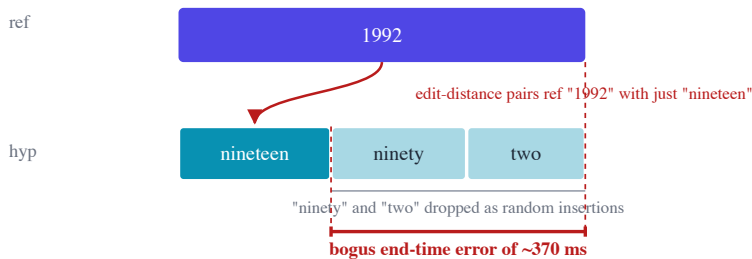
5. What this utterance contributes to the corpus metrics

This one utterance's local metrics: start MAE = $(77+27+36+19+72)/5 = 46$ ms · end MAE = $(27+36+19+72+318)/5 = 94$ ms. Notice how a *single* bad boundary ("all" end) doubles the end-MAE — this is exactly why we also report **median** (would have been 36 ms — much tighter) and **p90** (catches the outlier). The corpus-level metric is computed by pooling all word-pair offsets across every utterance, then computing MAE/median/p90/buckets once on the pooled set.

The multi-word substitution merge, visualized

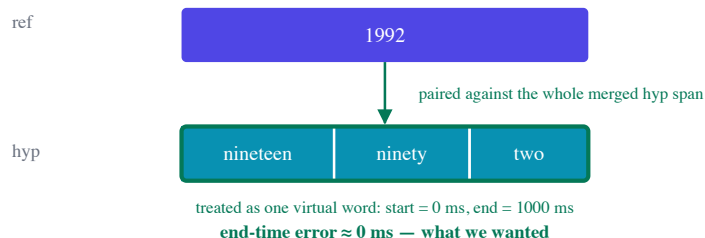
One word in the reference can map to multiple words in the hypothesis (or vice versa). The classic case is **inverse text normalization**: the reference transcript says "1992" but the provider, with `format_text` on, transcribed it as "nineteen ninety two". Without handling this, the harness would compare the timestamp of "1992" to the timestamp of just "nineteen", treating the other two words as random insertions — which produces *fake* timestamp errors.

Without merging (wrong)



Naïve edit-distance alignment pairs "1992" with just the first hypothesis word ("nineteen"), and treats "ninety" and "two" as random insertions that get dropped. The harness then compares the end-time of "1992" to the end-time of "nineteen", and concludes the model got the boundary 370 ms wrong — but the model didn't get anything wrong; it just spelled the number out.

With merging (right)



Whenever the alignment step finds an N-to-M substitution span, the harness merges the larger side into a single virtual word whose start is the first sub-word's start and whose end is the last sub-word's end. Now the comparison measures the full span the model emitted for that reference word, which is what we wanted in the first place.

What each metric actually means

MAE (mean absolute error)

What: the average distance — in milliseconds — between where the provider says a word starts/ends and where MFA says it does.

Why it can mislead: a single very-bad outlier pulls the average up a lot. The worked example above had four good word ends and one bad one, and MAE jumped from "tight" to "94 ms" because of that single outlier.

Median

What: the typical case — half the words are tighter than this, half are looser.

Why it matters: median is robust to outliers. If MAE is much bigger than median, the model has a few bad cases dragging the average up. That's a different problem than "the model is consistently a little off," and it calls for different fixes.

p90 (90th percentile)

What: the worst case, almost. 90% of words land tighter than this; 10% land looser.

Why it matters: if you're shipping a feature that depends on timestamps — subtitles, end-of-turn detection, voice-over editing — the worst 10% of words is what your users will notice. p90 is the realistic upper bound on "how bad will this feel."

% within X ms (tolerance buckets)

What: the fraction of words where the boundary lands inside a given tolerance. We report 25, 50, 100, and 200 ms.

Why it matters: these map directly to product decisions. Around 50 ms is roughly the limit of perceptual relevance for subtitle timing. 100 ms is a comfortable margin for response-trigger features. 200 ms is generous — anything outside that is "looks broken." Pick the bucket that matches what you're building.

Reading the tolerance buckets carefully

The buckets are **cumulative, not exclusive**. "% within 200 ms" means "the fraction of word boundaries that landed *anywhere from 0 to 200 ms* off the reference," not "the fraction in the 100–200 ms range specifically." A boundary that's only 30 ms off counts in the ≤ 50 , ≤ 100 , and ≤ 200 buckets simultaneously.

Concrete example using the LibriSpeech start-time numbers:

Bucket	AAI U3 Pro	Whisper-1	What it actually means
≤ 25 ms	18.9 %	18.5 %	fraction of starts within 25 ms of the reference
≤ 50 ms	43.2 %	41.0 %	within 50 ms (includes the ≤ 25 ms boundaries too)
≤ 100 ms	79.2 %	68.3 %	within 100 ms (includes both buckets above)
≤ 200 ms	99.3 %	83.9 %	within 200 ms (includes all of the above)

So "AAI wins at 200 ms" doesn't mean AAI is more often *in* the 100–200 ms range. It means AAI has fewer boundaries that miss by *more than* 200 ms — fewer catastrophic outliers. AAI's 99.3 % within 200 ms says only **0.7 %** of its word starts miss by more than 200 ms. Whisper's 83.9 % means **16.1 %** miss by more than 200 ms — about 22× as many catastrophically-bad starts. That's where Whisper's long p90 tail (310 ms) shows up in the bucket view.

Reading multiple buckets together tells the story

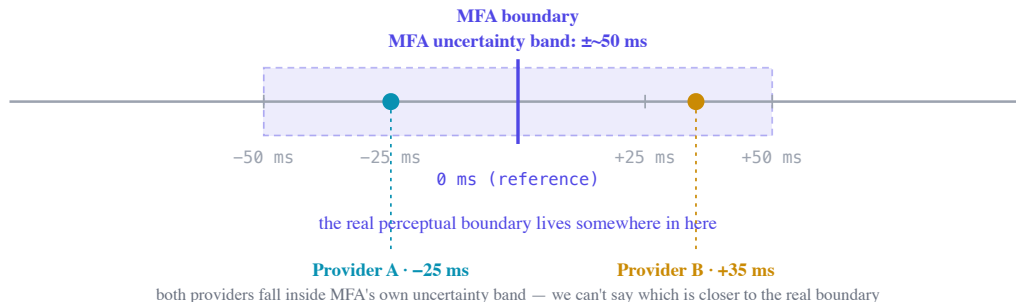
- **Wins at every bucket** → consistently better across the whole distribution.
- **Wins at tight (≤ 25 , ≤ 50) but loses at loose (≤ 200)** → very precise when it nails the timestamp, but has a long tail of catastrophic misses. (This is Whisper's pattern on peoples_speech — best ≤ 25 ms percentage, worst ≤ 200 ms.)
- **Wins at loose (≤ 200) but loses at tight (≤ 25)** → rarely catastrophic but rarely very precise either. Consistent, never great, never terrible.

If you actually want "% in the 100–200 ms range"

Subtract: `within_200` – `within_100` . For AAI on LibriSpeech, that's $99.3 - 79.2 = \mathbf{20.1\%}$ of boundaries fell strictly between 100 and 200 ms off. For Whisper, it's $83.9 - 68.3 = \mathbf{15.6\%}$. Useful if you care about a specific tolerance band rather than "anything tighter than X."

How small a difference is real?

MFA itself has alignment error. On clean read speech that's typically ~30–50 ms per boundary; on noisier or spontaneous speech, more. That's a **floor** on what these absolute numbers can resolve. Schematically:



The MFA boundary is the only point we actually know. The "real" perceptual boundary is somewhere in the dashed band, but we don't know exactly where. If two providers both land inside this band — even if they disagree with each other by 60 ms — we cannot conclude one is more accurate. The signal is below the noise floor of the reference. To resolve sub-30 ms differences you'd need a hand-annotated gold reference (or several MFA runs averaged with different acoustic models). **Rule of thumb for this corpus: treat MAE differences under ~30 ms as "tied."**

What this eval can and can't tell you

Can tell you:

- Which provider's word boundaries match MFA's most consistently on this corpus.
- How the distribution of timestamp errors looks (tight vs. long-tailed).
- Whether the provider's timestamps are good enough for a downstream feature with a known tolerance budget.
- Relative performance across providers, fairly — because all providers are compared against the same reference.

Can't tell you:

- Absolute "ground-truth" accuracy in ms — you're bounded below by MFA's own error.
- How a provider will do on speech that's wildly different from this corpus (noisy, accented, conversational, non-English) — you'd run it on those datasets specifically.
- Whether the model recognised the right words. This is a timestamp eval, not a WER eval. The match-rate column tells you roughly what fraction of reference words were aligned, but for word-recognition quality you'd use a WER harness.

Glossary

Forced alignment

Given audio and a known transcript, computing exactly when each word/phoneme occurs in the audio. "Cheats" in the sense that it has the right transcript to start with — that's why MFA's output is a stronger reference than a recogniser's output.

MFA (Montreal Forced Aligner)

The open-source, widely-used forced aligner this harness uses for reference timestamps. [docs](#)

TextGrid

The file format MFA outputs — one file per audio clip, listing each word with its start and end times.

Word-level timestamps

For each word the STT model emitted, the start and end time in the audio. Some providers produce them natively from the recognition model; others run a separate alignment pass on the recognised text. Quality varies; that's what we're measuring.

Inverse text normalization (ITN)

The post-processing step that turns spoken numbers / dates / addresses into their written form:

"twenty twenty six" → "2026", "five dollars" → "\$5". It's what makes transcripts look professional and also what creates N-to-M alignment mismatches in this eval.

Edit distance / minimum-edit-distance alignment

The same algorithm WER uses: find the cheapest sequence of inserts, deletes, and substitutions that turns one word list into the other. The resulting alignment tells us which hypothesis word corresponds to which reference word.

Substitution merging

This harness's fix for N-to-M alignment chunks. When a reference word maps to several hypothesis words (or vice versa), merge the longer side into one virtual word spanning from the first sub-word's start to the last sub-word's end, then compare timestamps as a 1:1 pair.

MAE / median / p90

Three different ways to summarise a distribution of errors: the average, the typical, and the worst-realistic. We report all three because they tell different stories.

Tolerance bucket

"% of word boundaries that landed within X milliseconds of the reference." More directly useful for product decisions than averages.

Pooled aggregation

Computing the corpus-level metric by pooling every word-pair offset across every utterance into one big set, then computing once. This is the correct way to compute corpus-level percentiles — a weighted mean of per-file medians is not the same as the median of the pool.

p90 tail

The shape of the distribution past the 90th percentile. A long p90 tail means "the bad cases are very bad," which is often more annoying to users than "everything is consistently OK."